MEDIUM3

Step1

任务1.1

1.绝对路径：从/（根目录）开始的路径（/var/backups/turing/是一个绝对路径）

相对路径：从当前目录开始，.表示当前目录,..表示上一级，每多一个../就继续向上一级

2.绝对路径：cd /var/backups/turing/

相对路径：cd ../../../var/backups/turing/\

任务1.2

1.argc表示读取的字符个数，argv[]表示把字符串存入数组

2.（如果上述命令指cd /var/backups/turing/）

argc=2，argv[0]=程序名文件名，argv[1]=cd，argv[2]=/var/backups/turing/

Step2

任务2.1

1.见过的：#include <stdio.h>标准输入输出函数

#include <string.h>字符串处理

#include <stdlib.h>内存分配时用到的工具函数

没见过：#include <dirent.h>（查）目录项操作，提供目录遍历相关类型函数，ls类核心头文件

#include <sys/stat.h>（查）文件状态与元数据，获取文件元数据结构体函数

#include <sys/types.h> 系统数据类型定义各种类型别名

#include <time.h>时间处理

2.<dirent.h>：类型DIR struct diren

函数opendir(),readdir(),closedir()。

用于打开关闭目录，遍历目录想，是实现ls目录遍历程序的基础

<sys/stat.h>：类型struct stat，mode_t

函数:stat(),lstat(),fstat()

获取文件元数据,inode 编号、文件类型、权限、大小、时间.提供文件类型判断宏。

2.2 void parse_permissions(mode_t mode, char *perms)

{

```
// 权限字符对照表
char rwx[] = {'r', 'w', 'x'};
// 请补全代码写在 TO DO 与 END OF TO DO 之间：
// TO DO
mode=mode&0777;//(即为111111111)
for(int i=0;i<9;i++)
{
    if(mode & (0400 >> i))//判断权限
    {
        perms[i]= rwx[i % 3] ;
    }
    else
    {
        perms[i]='-';
    }
}
// END OF TO DO
perms[9] = '\0'; // 字符串结尾
```

}

Step3

任务3.1

1.存储：1.文件大小2.文件权限3.文件类型4.文件创建时间5.文件数据存储位置

2.不包含1.文件名2.文件内容

3. .表示当前目录,..表示上一级目录

特殊规律：在根目录 / 下，. 和 .. 都指向根目录自身，它们的 inode 编号相同，且等于根目录的 inode 编号。

任务3.2

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/stat.h>

#include <dirent.h>

ino_t get_inode(const char *path)

{ struct stat statbuf;

```
  if (stat(path, &statbuf) == -1)

  {

      perror("stat failed");

      exit(EXIT_FAILURE);

  }

  return statbuf.st_ino;
```

}

void print_path(ino_t cur_ino)

{

```
  ino_t parent_ino = get_inode("..");//查上一级inode

  if (cur_ino == parent_ino) //判断是否到根目录

  {

      return;

  }

  if (chdir("..") == -1) //判断是否正常

  {

      perror("chdir ..");
```

```
    exit(EXIT_FAILURE);
}
DIR *dir = opendir(".");//打开并存储当前文件夹
if (!dir) //判断是否打开
{
    perror("opendir");
    exit(EXIT_FAILURE);
}
struct dirent *entry;//暂存
char name[256];
int found = 0;

while ((entry = readdir(dir)) != NULL)
{
    // 跳过 "." 和 "..", 防死循环
    if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
    {
        continue;
    }

    if (get_inode(entry->d_name) == cur_ino)
    {
        strncpy(name, entry->d_name, sizeof(name) - 1);
        name[sizeof(name) - 1] = '\0';//
        found = 1;
        break;
    }
    if(found==0)
```

```
        {
            return;
        }
        closedir(dir);
        print_path(parent_ino);
        printf("/%s",name);//到根目录开始逐级打印
    }
```

}

int main()

{

```
    ino_t cur_ino = get_inode(".");     // 获取当前目录的 inode
    ino_t parent_ino = get_inode(".."); // 获取父目录的 inode

    if (cur_ino == parent_ino)
    {
        printf("/\n"); // 如果一开始就在根目录，打印/
    }
    else
    {
        print_path(cur_ino); // 开始往上倒推
        printf("\n");
    }
    return 0;
```

}

Step4

任务4.1

```
#include <stdio.h>

#include <string.h>

#include <dirent.h>

#include <sys/stat.h>

#include <unistd.h>

// 递归查找指定文件，若找到则打印其相对路径

void find_file(const char *current_path, const char *target_name)

{

    DIR *dir;

    struct dirent *entry;

    struct stat statbuf;


    if (!(dir = opendir(current_path)))

    {

        return; // 无法打开目录则跳过

    }

    while ((entry = readdir(dir)) != NULL)

    {

        // 忽略当前目录 . 和父目录 .. 防止无限死循环

        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)

        {

            continue;

        }

        char full_path[1024];//full_file为current_file+entry->d_name

        snprintf(full_path, sizeof(full_path), "%s/%s", current_path, entry->d_name);

        if (stat(full_path, &statbuf) == -1) continue;
```

```
        // 请补全代码写在 TO DO 与 END OF TO DO 之间：
        // 要求：
        // 1. 检查当前 entry 的文件名是否与 target_name 匹配，若匹配则打印 full_path。
        // 2. 如果当前 entry 是一个目录，则需要对其进行深度优先的递归查找。
        // （提示：若(statbuf.st_mode & S_IFMT) == S_IFDIR则statbuf为目录文件）
        // TO DO
        if(strcmp(entry->d_name,target_name)==0)
        {
            printf("%s\n",full_path);
        }
        if((statbuf.st_mode & S_IFMT) == S_IFDIR)
        {
            find_file(full_path,target_name);
        }
        // END OF TO DO
    }
    closedir(dir);
}
```